

Architecture Overview — Family Music Scrobbler PWA

Architecture Overview - Family Music Scrobbler PWA

A high-level architectural overview of the Family Music Scrobbler PWA, detailing system components, data flow, deployment topology, and operational boundaries across development and production environments.

1. System Goals

The Family Music Scrobbler PWA is designed to:

- Collect and store long-term Spotify listening history for multiple family members.
 - Provide a fast, offline-capable PWA interface.
 - Deliver analytics such as recent tracks, weekly leaderboards, and profile insights.
 - Operate reliably in a home-lab environment using Docker and a Fedora Server-based Intel NUC.
 - Maintain secure token storage, robust ingestion, and verifiable backups.
-

2. High-Level Architecture

The system consists of four major subsystems:

1. Frontend (React PWA)

- React application served as a Progressive Web App.
- Service Worker handles caching and offline behaviour.
- IndexedDB stores offline sync queue and cached track data.
- Background Sync API pushes queued updates when connectivity returns.
- Push API delivers leaderboard notifications.

2. Backend (FastAPI)

- Exposes REST API endpoints for authentication, track ingestion, user profiles, and analytics.
- Hosts APScheduler worker for Spotify polling.
- Manages encrypted refresh tokens.
- Provides /healthz endpoint for observability.

3. Database (PostgreSQL)

- Stores users, tracks, scrobbles, ingestion metadata, and tokens.
- Monthly partitioning of scrobbles table for performance.
- Enforces deduplication via `UNIQUE(user_id, spotify_play_id)`.
- Indexed for fast time-range queries.

4. Deployment & Infrastructure

- Docker Compose orchestrates backend, frontend, and database.
 - Production runs on Intel NUC (Fedora Server).
 - Backups stored on Synology DS223 via rsync.
 - Secrets stored using Docker secrets.
-



3. Data Flow Overview

1. Spotify → Backend Ingestion Worker

- APScheduler triggers polling every 5-10 minutes.
- Worker calls Spotify user-read-recently-played.
- Response is deduplicated using `spotify_play_id`.
- New scrobbles inserted into partitioned tables.
- `last_polled` timestamp updated.

2. Backend → Frontend API

- Frontend requests recent tracks, leaderboard data, and profile info.
- Network-first strategy ensures fresh data when online.
- Cached responses used when offline.

3. Frontend Offline Sync

- User actions (e.g., profile edits) stored in IndexedDB when offline.
- Background Sync API retries until successful.

4. Backend → Synology Backup

- `verify_backup.sh` performs `pg_dump`.
- Dump synced to NAS.

- Temporary container restores dump for integrity verification.
-

4. Database Architecture

Partitioning Strategy

The scrobbles table is partitioned by month:

```
scrobbles_2026_09
scrobbles_2026_10
scrobbles_2026_11
...
```

This ensures:

- Efficient pruning of old partitions.
- Fast time-range queries.
- Reduced index bloat.

Key Constraints

- played_at TIMESTAMPTZ NOT NULL
- UNIQUE(user_id, spotify_play_id)
- Foreign keys to tracks and users tables.

Indexes

- (user_id, played_at) for chronological queries.
 - (spotify_play_id) for deduplication.
-

5. Security Architecture

Token Security

- Spotify refresh tokens encrypted using AES.
- AES key stored in Docker secrets.
- Tokens decrypted only in worker runtime.

PWA Security

- HTTPS enforced.
- Service Worker restricted to app scope.
- VAPID keys stored securely.

Backend Security

- Strict CORS rules.
 - Rate limiting on ingestion endpoints.
 - Structured JSON logging for auditability.
-

6. Deployment Architecture

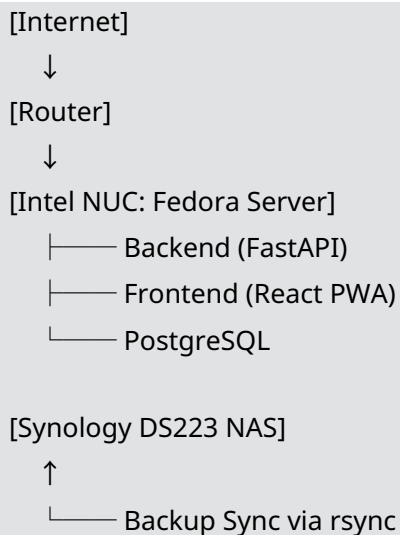
Development (Fedora 43)

- Local Docker Compose with override file.
- Hot reload for backend and frontend.
- GitHub Actions CI pipeline.

Production (Intel NUC)

- Production Docker Compose.
- Secrets mounted from /opt/family-scrobblerr/secrets/.
- PostgreSQL volume stored on SSD.
- Backups synced to Synology DS223.

Network Topology



7. Observability & Monitoring

Health Checks

- /healthz endpoint reports:

- DB connectivity
- Worker heartbeat
- API availability

Logging

- JSON logs for ingestion worker.
- API request logs.
- Error logs stored in Docker volumes.

Backup Verification

- Automated via `verify_backup.sh`.
- Ensures recoverability.



8. Architectural Principles

- **Offline-first** PWA design.
- **Secure-by-default** token handling.
- **Partitioned storage** for long-term scalability.
- **Home-lab friendly** deployment.
- **Strict separation** of dev and production environments.
- **Predictable ingestion** via APScheduler.



9. Summary

This architecture ensures:

- Reliable ingestion of Spotify history.
- Fast, offline-capable user experience.
- Secure token and data handling.
- Robust backup and recovery.
- Smooth deployment on home-lab hardware.

It provides a scalable foundation for future enhancements such as YouTube Music ingestion, advanced analytics, and multi-device sync.